

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 5**



Search

Disusun oleh :
Vivaldi Fachmy Fahlevi **NIM.2510817210017**

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 5

Laporan Praktikum Algoritma Dan Struktur Data Modul 5 : Search ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Vivaldi Fachmy Fahlevi
NIM : 2510817210017

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir.Muhmmad Alkaff, S,Kom, M.Kom, Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN	2
DAFTAR ISI.....	3
DAFTAR GAMBAR	4
DAFTAR TABEL.....	5
Searching.....	1
A. Source Code	1
B. Output Program	6
C. Pembahasan	7
Linear Search	21
Binary Search.....	25

DAFTAR GAMBAR

Gambar 1 Hasil Pengujian Linear Search	6
Gambar 2 Hasil Pengujian Binary Search.....	6

DAFTAR TABEL

Tabel 1 Source Code File searching_algorithms.h	1
Tabel 2 Source Code File searching_algorithms.cpp.....	1
Tabel 3 Source Code File metrics.h	2
Tabel 4 Source Code File reporter.h	3
Tabel 5 Source Code File reporter.cpp	3
Tabel 6 Source Code File data_generator.h	4
Tabel 7 Source Code File data_generator.cpp	4

Searching

A. Source Code

Tabel 1 Source Code File searching_algorithms.h

1	#include "searching_algorithms.h"
2	#include <chrono>
3	
4	using Clock = std::chrono::high_resolution_clock;
5	
6	void linear_search(const std::vector<int>& data,
7	int target, Metrics& m) {
8	auto start = Clock::now();

Tabel 2 Source Code File searching_algorithms.cpp

1	#include "searching_algorithms.h"
2	#include <chrono>
3	
4	using Clock = std::chrono::high_resolution_clock;
5	
6	void linear_search(const std::vector<int>& data,
7	int target, Metrics& m) {
8	auto start = Clock::now();
9	m.comparisons = 0;
10	m.found_index = -1;
11	
12	for (size_t i = 0; i < data.size(); i++) {
13	m.comparisons++;
14	
15	if (data[i] == target) {
16	m.found_index = i;
17	break;
18	}
19	}
20	
21	m.time_us = std::chrono::duration<long
22	double, std::micro>(Clock::now() - start
23	).count();
24	}
25	
26	void binary_search(const std::vector<int>& data,
27	int target, Metrics& m) {
28	auto start = Clock::now();

```

29     m.comparisons = 0;
30     m.found_index = -1;
31
32     int left = 0;
33     int right = data.size() - 1;
34
35     while (left <= right) {
36         int mid = (left + right) / 2;
37
38         m.comparisons++;
39
40         if (data[mid] == target) {
41             m.found_index = mid;
42             break;
43         }
44         else if (data[mid] < target) {
45             left = mid + 1;
46         }
47         else {
48             right = mid - 1;
49         }
50     }
51
52     m.time_us = std::chrono::duration<long
double, std::micro>(
53         Clock::now() - start
54     ).count();
55 }

```

Tabel 3 Source Code File metrics.h

```

1  #pragma once
2  #include <string>
3  #include <chrono>
4
5  struct Metrics {
6      std::string algorithm_name;
7      std::string input_type;
8      int n;
9
10     long long comparisons = 0;
11     long long found_index = -1; // -1 for not
found
12
13     long double time_us = 0.0;
14
15     void reset() {
16         comparisons = 0;
17         found_index = -1;

```

18	time_us = 0.0;
19	}
20	};
21	

Tabel 4 Source Code File reporter.h

1	#pragma once
2	#include "metrics.h"
3	
4	void print_table_header();
5	void print_metrics(const Metrics& m);
6	void print_separator();

Tabel 5 Source Code File reporter.cpp

1	#include "reporter.h"
2	#include <iostream>
3	#include <iomanip>
4	
5	const int W_ALG = 16;
6	const int W_CASE = 10;
7	const int W_N = 10;
8	const int W_CMP = 14;
9	const int W_IDX = 14;
10	const int W_TIME = 16;
11	
12	void print_separator() {
	std::cout << std::string(W_ALG + W_CASE + W_N
13	+ W_CMP + W_IDX + W_TIME, '-') << "\n";
14	}
15	
16	void print_table_header() {
17	print_separator();
18	std::cout << std::left
19	<< std::setw(W_ALG) << "Algorithm"
20	<< std::setw(W_CASE) << "Case"
21	<< std::setw(W_N) << "N"
	<< std::setw(W_CMP) <<
22	"Comparisons"
	<< std::setw(W_IDX) << "Found
23	Index"
24	<< std::setw(W_TIME) << "Time(us)"
25	<< "\n";
26	print_separator();
27	}
28	
29	void print_metrics(const Metrics& m) {
30	std::string found = (m.found_index == -1)

31	? "Not Found"
	:
32	std::to_string(m.found_index);
33	
34	std::cout << std::left
	<< std::setw(W_ALG) <<
35	m.algorithm_name
	<< std::setw(W_CASE) <<
36	m.input_type
37	<< std::setw(W_N) << m.n
	<< std::setw(W_CMP) <<
38	m.comparisons
39	<< std::setw(W_IDX) << found
	<< std::fixed <<
40	std::setprecision(3)
41	<< std::setw(W_TIME) << m.time_us
42	<< "\n";
43	}
44	

Tabel 6 Source Code File data_generator.h

1	#pragma once
2	#include <vector>
3	#include <string>
4	
5	enum class SearchCase {
6	BEST,
7	AVERAGE,
8	WORST
9	};
10	
11	std::string search_case_name(SearchCase sc);
12	
13	std::vector<int> generate_sorted_data(int n);
14	
15	int generate_target(const std::vector<int>& data,
16	SearchCase sc,
17	bool is_binary,
18	unsigned int seed = 42);
19	

Tabel 7 Source Code File data_generator.cpp

1	#include "data_generator.h"
2	#include <numeric>
3	#include <random>
4	#include <stdexcept>
5	

```

6  std::string search_case_name(SearchCase sc) {
7      switch (sc) {
8          case SearchCase::BEST:      return
9  "Best      ";
10         case SearchCase::AVERAGE: return
11 "Average";
12         case SearchCase::WORST:     return
13 "Worst     ";
14         default:                     return
15 "Unknown";
16     }
17 }
18
19 std::vector<int> generate_sorted_data(int n) {
20     if (n <= 0) throw std::invalid_argument("N
21 harus > 0");
22     std::vector<int> data(n);
23     std::iota(data.begin(), data.end(), 1);
24     return data;
25 }
26
27 int generate_target(const std::vector<int>& data,
28                     SearchCase sc,
29                     bool is_binary,
30                     unsigned int seed)
31 {
32     int n = (int)data.size();
33
34     switch (sc) {
35         case SearchCase::BEST:
36             return is_binary ? data[n / 2] :
37 data[0];
38
39         case SearchCase::AVERAGE: {
40             std::mt19937 rng(seed);
41             std::uniform_int_distribution<int>
42 dist(0, n - 1);
43             return data[dist(rng)];
44         }
45
46         case SearchCase::WORST:
47             return data[n - 1] + 9999;
48
49         default:
50             return -1;
51     }
52 }

```

B. Output Program

```
>> Linear Search
```

Algorithm	Case	N	Comparisons	Found Index	Time(us)
Linear Search	Best	1000	1	0	0.100
Linear Search	Average	1000	375	374	2.700
Linear Search	Worst	1000	1000	Not Found	4.500
Linear Search	Best	10000	1	0	0.100
Linear Search	Average	10000	3746	3745	16.800
Linear Search	Worst	10000	10000	Not Found	45.800
Linear Search	Best	100000	1	0	0.100
Linear Search	Average	100000	37455	37454	162.700
Linear Search	Worst	100000	100000	Not Found	439.900
Linear Search	Best	1000000	1	0	0.100
Linear Search	Average	1000000	374541	374540	2043.900
Linear Search	Worst	1000000	1000000	Not Found	7421.300

Gambar 1 Hasil Pengujian Linear Search

```
>> Binary Search
```

Algorithm	Case	N	Comparisons	Found Index	Time(us)
Binary Search	Best	1000	9	500	0.200
Binary Search	Average	1000	3	374	0.100
Binary Search	Worst	1000	10	Not Found	0.400
Binary Search	Best	10000	13	5000	0.200
Binary Search	Average	10000	13	3745	0.300
Binary Search	Worst	10000	14	Not Found	0.500
Binary Search	Best	100000	16	50000	0.200
Binary Search	Average	100000	16	37454	0.400
Binary Search	Worst	100000	17	Not Found	0.500
Binary Search	Best	1000000	19	500000	0.300
Binary Search	Average	1000000	15	374540	1.100
Binary Search	Worst	1000000	20	Not Found	0.400

Gambar 2 Hasil Pengujian Binary Search

C. Pembahasan

Pada bagian file **searching_algorithms.cpp** dilakukan `#include` terhadap file **searching_algorithms.h** dan library `<chrono>` pada baris [1] dan [2]. File header digunakan agar deklarasi fungsi dapat dikenali oleh program lain, sedangkan library `<chrono>` digunakan untuk mengukur waktu eksekusi algoritma. Kemudian pada baris [4], dibuat alias `Clock` untuk `std::chrono::high_resolution_clock`. Tujuan penggunaan alias ini adalah agar penulisan kode menjadi lebih singkat dan mudah dibaca tanpa harus menuliskan nama yang panjang berulang kali.

Fungsi `linear_search` dimulai pada baris [6]. Pada baris [7], program menyimpan waktu awal eksekusi menggunakan `Clock::now()`. Langkah ini diperlukan karena program ingin menghitung berapa lama proses pencarian berlangsung. Setelah itu pada baris [9] dan [10], nilai `comparisons` diatur menjadi 0 dan `found_index` diatur menjadi -1. Nilai 0 digunakan karena sebelum pencarian dimulai belum ada `comparison` yang dilakukan, sedangkan -1 digunakan sebagai tanda awal bahwa data belum ditemukan.

Proses utama `linear search` terjadi pada `Loop for` di baris [12]. Algoritma memeriksa setiap elemen mulai dari indeks pertama hingga indeks terakhir. Pada baris [13], setiap kali satu elemen diperiksa maka `comparisons` ditambah satu. Penambahan ini dilakukan agar program dapat mengetahui berapa banyak perbandingan yang terjadi selama proses pencarian. Kemudian pada baris [15], program membandingkan isi array dengan nilai target. Jika nilai tersebut sama, maka indeks tempat data ditemukan disimpan ke `found_index` pada baris [16]. Setelah data ditemukan, program langsung menghentikan `loop` menggunakan `break` pada baris [17]. Penggunaan `break` penting karena setelah target ditemukan, tidak ada alasan untuk memeriksa elemen berikutnya. Dengan menghentikan `loop` lebih awal, jumlah operasi dapat dikurangi.

Cara kerja `linear search` dapat dibuktikan langsung dari alur *loop*-nya. Misalnya terdapat data [10, 20, 30, 40] dan target 30. Program pertama memeriksa 10, lalu 20, kemudian 30. Ketika nilai 30 ditemukan, indeks 2 disimpan ke `found_index` dan `loop` berhenti.

Pada baris [21]-[23], program menghitung waktu eksekusi menggunakan selisih waktu sekarang dengan waktu awal yang disimpan sebelumnya. Hasilnya

disimpan dalam satuan ms (microseconds) ke variabel `time_us`. Langkah ini dilakukan agar program dapat membandingkan kecepatan masing-masing algoritma secara lebih jelas.

Fungsi `binary_search` dimulai pada baris [26]. Bagian awal fungsi hampir sama dengan `linear search`, yaitu menyimpan waktu awal eksekusi pada baris [27], lalu mengatur nilai awal `comparisons` dan `found_index` pada baris [29] dan [30]. Perbedaannya mulai terlihat pada penggunaan variabel `left` dan `right` di baris [32] dan [33]. Kedua variabel ini digunakan untuk menentukan batas kiri dan kanan area pencarian. Ide utama `binary search` adalah mempersempit area pencarian sedikit demi sedikit hingga target ditemukan.

Pada baris [35], program menggunakan loop `while` yang akan terus berjalan selama batas kiri masih lebih kecil atau sama dengan batas kanan. Di dalam loop, program menghitung indeks tengah menggunakan rumus pada baris [36]. Nilai tengah ini sangat penting karena `binary search` selalu memulai pemeriksaan dari elemen tengah, bukan dari awal data seperti `linear search`.

Pada baris [40], `comparison` kembali ditambah satu setiap kali elemen tengah diperiksa. Kemudian pada baris [42], program memeriksa apakah nilai tengah sama dengan target. Jika sama, maka indeks tengah disimpan ke `found_index` pada baris [43] dan pencarian dihentikan menggunakan `break` pada baris [44].

Jika nilai tengah tidak sama dengan target, program akan menentukan arah pencarian berikutnya. Pada baris [46]-[48], jika nilai tengah lebih kecil dari target maka batas kiri dipindahkan ke $\text{mid} + 1$. Hal ini dilakukan karena target pasti berada di sebelah kanan data tengah. Sebaliknya pada baris [50]-[52], jika nilai tengah lebih besar dari target maka batas kanan dipindahkan ke $\text{mid} - 1$ karena target pasti berada di sebelah kiri. Mekanisme inilah yang membuat `binary search` jauh lebih cepat dibandingkan `linear search` karena setiap langkah langsung membuang setengah area pencarian.

Sebagai contoh, jika data berisi [10, 20, 30, 40, 50, 60, 70] dan target yang dicari adalah 60, maka program pertama memeriksa nilai tengah 40. Karena 60 lebih besar dari 40, maka bagian kiri langsung diabaikan dan pencarian dilanjutkan

hanya pada bagian kanan. Selanjutnya nilai tengah akan diperiksa hingga akhirnya 60 ditemukan. Binary search tidak perlu memeriksa seluruh data satu per satu.

Fungsi binary search di baris [56]-[58], waktu eksekusi kembali dihitung menggunakan cara yang sama seperti linear search. Dengan adanya perhitungan waktu dan comparison.

File **metrics.h** berfungsi sebagai tempat penyimpanan hasil pengujian algoritma pencarian. Struct Metrics digunakan agar semua informasi yang berkaitan dengan proses pengujian dapat dikumpulkan dalam satu tempat sehingga lebih mudah digunakan dan dikelola oleh program lain.

Pada baris [1], digunakan `#pragma once` untuk mencegah file header dimuat berulang kali oleh compiler. Hal ini penting agar tidak terjadi duplikasi definisi ketika file dipanggil di beberapa bagian program.

Pada baris [2] dan [3], program memanggil library yang diperlukan untuk mendukung penggunaan tipe data string dan pengukuran waktu. Library tersebut dibutuhkan karena struct menyimpan nama algoritma, jenis input, dan hasil waktu eksekusi algoritma.

Struct dimulai pada baris [6]. Struct dipilih karena program membutuhkan beberapa variabel yang saling berkaitan untuk menyimpan hasil pengujian. Dengan menggunakan struct, semua data dapat dikumpulkan menjadi satu objek sehingga lebih rapi dibanding membuat banyak variabel terpisah.

Pada baris [7] dan [8], program menyimpan informasi mengenai nama algoritma dan jenis pengujian yang dilakukan. Data ini penting karena program melakukan beberapa kondisi pengujian seperti best case, average case, dan worst case sehingga hasilnya perlu dibedakan.

Pada baris [9], terdapat variabel yang digunakan untuk menyimpan ukuran data yang diuji. Informasi ini diperlukan karena performa algoritma sangat dipengaruhi oleh banyaknya data yang diproses.

Pada baris [11], program menyimpan jumlah comparison yang dilakukan selama pencarian. Nilai awal dibuat 0 karena sebelum algoritma berjalan belum ada comparison yang terjadi. Tipe data yang digunakan dipilih agar mampu menampung jumlah comparison yang sangat besar ketika ukuran data meningkat.

Pada baris [12], program menyimpan indeks tempat target ditemukan. Nilai awal diberikan -1 sebagai penanda bahwa target belum ditemukan. Nilai ini dipilih karena indeks array normal dimulai dari 0, sehingga -1 dapat digunakan sebagai tanda khusus bahwa data tidak ada di dalam array.

Pada baris [14], program menyimpan waktu eksekusi algoritma dalam satuan microseconds. Tipe data yang digunakan dibuat lebih presisi karena waktu

eksekusi algoritma sangat kecil sehingga membutuhkan satuan pengukuran yang cukup kecil.

Pada baris [16]-[20], terdapat fungsi untuk mengembalikan semua hasil pengujian ke kondisi awal. Fungsi ini penting agar hasil pengujian sebelumnya tidak mempengaruhi pengujian berikutnya. Tanpa proses reset, nilai comparison atau waktu sebelumnya masih bisa tersimpan sehingga hasil baru menjadi tidak akurat.

File **searching_algorithms.h** berfungsi sebagai file deklarasi fungsi algoritma pencarian. Pada baris [1], kembali digunakan `#pragma once` agar file tidak dimuat lebih dari satu kali. Pada baris [2], program memanggil library vector karena data pencarian disimpan dalam bentuk vector integer. Kemudian pada baris [3], file ini memanggil `metrics.h` karena kedua algoritma membutuhkan struct `Metrics` untuk menyimpan hasil pengujian.

Pada baris [6] dan [7], program mendeklarasikan fungsi `linear search` dan `binary search`. Kedua fungsi menerima tiga hasil dari data yang akan dicari, target yang ingin ditemukan, dan objek `metrics` untuk menyimpan hasil pengujian seperti jumlah `comparison`, indeks data yang ditemukan, dan waktu eksekusi algoritma.

File **reporter.h** dan **reporter.cpp** digunakan untuk menampilkan hasil pengujian algoritma pencarian dalam bentuk tabel yang rapi. Kedua file ini berfungsi sebagai bagian output program, yaitu menampilkan informasi seperti nama algoritma, jenis pengujian, jumlah comparison, indeks data yang ditemukan, dan waktu eksekusi.

Pada file **reporter.h**, baris [1] menggunakan `#pragma once` untuk mencegah file dimuat berulang kali oleh compiler. Kemudian pada baris [2], file memanggil `metrics.h` karena fungsi-fungsi pada `reporter` membutuhkan struct `Metrics` untuk mengambil data hasil pengujian algoritma.

Pada baris [4]-[6], program mendeklarasikan tiga fungsi yang akan digunakan untuk menampilkan tabel hasil pengujian. Fungsi pertama digunakan untuk menampilkan bagian header tabel, fungsi kedua digunakan untuk menampilkan isi data hasil pengujian, dan fungsi ketiga digunakan untuk membuat garis pemisah agar tampilan tabel lebih rapi dan mudah dibaca.

Pada file **reporter.cpp**, baris [1] memanggil file `reporter.h` agar deklarasi fungsi yang sudah dibuat dapat digunakan kembali. Kemudian pada baris [2] dan [3], program memanggil library yang digunakan untuk menampilkan output ke layar dan mengatur format tampilan tabel.

Pada baris [6]-[11], program membuat beberapa konstanta ukuran kolom tabel. Setiap konstanta digunakan untuk menentukan lebar masing-masing bagian tabel seperti kolom nama algoritma, jenis kasus, ukuran data, jumlah comparison, indeks data, dan waktu eksekusi. Tujuan penggunaan konstanta ini adalah agar tampilan tabel tetap rapi walaupun isi data memiliki panjang yang berbeda-beda. Tanpa pengaturan lebar kolom, hasil output bisa terlihat berantakan dan sulit dibaca.

Fungsi `print_separator()` dimulai pada baris [13]. Fungsi ini digunakan untuk menampilkan garis horizontal sebagai pembatas tabel. Pada baris [14], panjang garis dihitung dari penjumlahan seluruh lebar kolom yang sudah ditentukan sebelumnya. Setelah itu program menampilkan karakter “-” secara berulang sehingga menghasilkan garis pemisah. Proses ini membantu membuat tampilan tabel lebih terstruktur dan mudah dipahami.

Fungsi `print_table_header()` dimulai pada baris [17]. Pada baris [18], program memanggil fungsi pemisah terlebih dahulu agar bagian atas tabel memiliki garis pembatas. Kemudian pada baris [19]-[25], program menampilkan nama setiap kolom tabel seperti nama algoritma, jenis kasus, ukuran data, jumlah comparison, indeks data yang ditemukan, dan waktu eksekusi. Penggunaan format pada bagian ini bertujuan agar semua teks rata kiri dan berada pada posisi kolom yang sesuai. Setelah header selesai ditampilkan, pada baris [27] program kembali memanggil fungsi pemisah agar bagian header terpisah jelas dari isi tabel.

Fungsi `print_metrics()` dimulai pada baris [30]. Fungsi ini digunakan untuk menampilkan hasil pengujian yang tersimpan di dalam objek `Metrics`. Pada baris [31]-[33], program terlebih dahulu memeriksa nilai `found_index`. Jika nilainya -1, maka program menampilkan tulisan “Not Found”. Namun, jika target ditemukan, maka program mengubah nilai indeks menjadi string agar dapat ditampilkan ke layar.

Pada baris [35]-[42], program menampilkan seluruh hasil pengujian ke dalam tabel. Data seperti nama algoritma, jenis pengujian, ukuran data, jumlah comparison, indeks data, dan waktu eksekusi ditampilkan sesuai kolom masing-masing. Pada baris [41], program mengatur agar angka desimal waktu eksekusi ditampilkan hingga tiga angka di belakang koma.

File **data_generator.h** dan **data_generator.cpp** digunakan untuk membuat data pengujian yang akan dipakai oleh algoritma search. Kedua file ini berfungsi untuk menghasilkan kumpulan data terurut serta menentukan target pencarian sesuai kondisi pengujian seperti best case, average case, dan worst case.

Pada file **data_generator.h**, pada baris [2] dan [3], program memanggil library yang diperlukan untuk penggunaan vector dan string.

Pada baris [6]-[10], program membuat enum class bernama SearchCase. Enum ini digunakan untuk mewakili jenis kondisi pengujian, yaitu best case, average case, dan worst case. Penggunaan enum dipilih agar program lebih mudah dibaca dan mengurangi kemungkinan kesalahan penulisan dibanding menggunakan angka atau string secara langsung. Dengan enum, program dapat langsung memahami kondisi pengujian yang sedang digunakan.

Pada baris [12], program mendeklarasikan fungsi yang digunakan untuk mengubah nilai enum menjadi teks. Fungsi ini diperlukan agar nama kondisi pengujian dapat ditampilkan dengan mudah pada tabel hasil output.

Pada baris [14], program mendeklarasikan fungsi untuk membuat data terurut dengan ukuran tertentu. Data terurut diperlukan terutama untuk binary search karena algoritma tersebut hanya dapat bekerja dengan benar jika data sudah dalam keadaan terurut.

Pada baris [16]-[19], program mendeklarasikan fungsi untuk menentukan target pencarian berdasarkan kondisi pengujian yang dipilih. Fungsi ini menerima kumpulan data, jenis kasus pengujian, penanda apakah algoritma yang digunakan adalah binary search atau bukan, serta nilai seed untuk random generator. Tujuan adanya seed adalah agar hasil pengacakan dapat tetap konsisten ketika program dijalankan kembali.

Pada file **data_generator.cpp**, baris [1] memanggil file **data_generator.h** agar semua deklarasi fungsi dapat digunakan kembali. Kemudian pada baris [2]-[4], program memanggil library tambahan yang digunakan untuk mengisi data otomatis, membuat angka acak, dan menangani error.

Fungsi **search_case_name()** dimulai pada baris [7]. Fungsi ini menggunakan switch untuk menentukan teks yang sesuai dengan kondisi pengujian. Jika nilai enum adalah best case maka program mengembalikan teks “Best”, jika average case maka mengembalikan “Average”, dan jika worst case maka mengembalikan “Worst”. Pada baris [12], terdapat kondisi default yang mengembalikan “Unknown”. Bagian ini digunakan sebagai pengaman apabila nilai data enum yang diterima tidak sesuai dengan kondisi yang tersedia.

Fungsi **generate_sorted_data()** dimulai pada baris [15]. Pada baris [16], program terlebih dahulu memeriksa apakah ukuran data lebih kecil atau sama dengan nol. Jika kondisi tersebut terjadi, program akan menghasilkan error karena tidak masuk akal membuat data dengan ukuran nol atau negatif. Pemeriksaan ini dilakukan untuk menjaga agar program tetap berjalan dengan benar dan menghindari data yang tidak valid.

Pada baris [17], program membuat vector dengan ukuran sesuai nilai *n*. Kemudian pada baris [18], program menggunakan fungsi untuk mengisi vector secara berurutan mulai dari angka 1 hingga *n*. Proses ini membuat data menjadi terurut secara otomatis. Sebagai contoh, jika $n = 5$, maka data yang dihasilkan adalah [1, 2, 3, 4, 5]. Cara ini dipilih karena lebih singkat dan lebih efisien dibanding mengisi data menggunakan loop manual.

Fungsi **generate_target()** dimulai pada baris [22]. Fungsi ini bertugas menentukan target pencarian berdasarkan kondisi pengujian. Pada baris [27]-[28], jika kondisi yang dipilih adalah best case, maka target akan disesuaikan dengan jenis algoritma yang digunakan. Jika algoritmanya binary search, maka target diambil dari elemen tengah data karena binary search memulai pencarian dari bagian tengah. Sedangkan jika algoritmanya linear search, target diambil dari elemen pertama karena linear search memulai pemeriksaan dari awal data. Pemilihan target seperti ini dilakukan agar benar-benar menghasilkan kondisi best case pada masing-masing algoritma.

Pada baris [30]-[34], program menangani kondisi average case. Di bagian ini program menggunakan random generator untuk memilih salah satu elemen data secara acak sebagai target pencarian. Proses ini dilakukan karena average case bertujuan menggambarkan kondisi pencarian umum yang tidak selalu berada di awal atau akhir data. Dengan memilih target secara acak, posisi target bisa berada di berbagai bagian data sehingga hasil pengujian menjadi lebih realistis.

Pada baris [36]-[37], program menangani kondisi worst case. Target dibuat dengan mengambil nilai terakhir data lalu menambahkan angka besar sehingga target dipastikan tidak ada di dalam data. Sebagai contoh, jika data terakhir bernilai 1000, maka target menjadi 10999. Cara ini dipilih agar algoritma benar-benar melakukan pemeriksaan maksimal sebelum menyimpulkan bahwa data tidak ditemukan.

Pada baris [39]-[40], program memberikan nilai -1 sebagai kondisi default apabila jenis pengujian tidak dikenali. Bagian ini digunakan sebagai pengaman tambahan agar fungsi tetap mengembalikan nilai walaupun kondisi yang diterima tidak sesuai.

File **main.cpp** merupakan program utama yang digunakan untuk menjalankan pengujian algoritma search. Program ini bertugas mengatur seluruh proses pengujian mulai dari menentukan ukuran data, memilih jenis kasus pengujian, menjalankan algoritma pencarian, hingga menampilkan hasilnya ke dalam bentuk tabel. Dengan kata lain, file ini menjadi pusat pengendali yang menghubungkan seluruh file sebelumnya seperti `metrics`, `data_generator`, `searching_algorithms`, dan `reporter`.

Pada baris [1]-[4], program memanggil beberapa library standar C++. Library tersebut digunakan untuk menampilkan output, menyimpan kumpulan data, menggunakan fungsi sebagai variabel, dan mendukung penggunaan string. Setiap library dipilih sesuai kebutuhan program agar fitur-fitur yang digunakan dapat berjalan dengan benar.

Pada baris [6]-[9], program memanggil file header yang sebelumnya sudah dibuat. File `metrics.h` digunakan untuk menyimpan hasil pengujian, `data_generator.h` digunakan untuk membuat data dan target pencarian, `searching_algorithms.h` berisi algoritma pencarian, sedangkan `reporter.h` digunakan untuk menampilkan hasil pengujian dalam bentuk tabel. Pemisahan file seperti ini dilakukan agar setiap bagian program memiliki tugas masing-masing sehingga struktur kode menjadi lebih rapi dan mudah dipahami.

Pada baris [12], program membuat kumpulan ukuran data yang akan diuji. Nilai yang digunakan adalah 1000, 10000, 100000, dan 1000000. Pemilihan ukuran data yang terus meningkat dilakukan agar program dapat melihat bagaimana performa algoritma berubah ketika jumlah data semakin besar. Dengan cara ini, perbedaan efisiensi linear search dan binary search dapat terlihat lebih jelas.

Pada baris [14]-[18], program menentukan jenis kasus pengujian yang akan digunakan, yaitu best case, average case, dan worst case. Ketiga kondisi ini dipilih karena masing-masing menggambarkan situasi pencarian yang berbeda. Best case menunjukkan kondisi terbaik, average case menunjukkan kondisi umum, sedangkan worst case menunjukkan kondisi paling sial bagi algoritma.

Pada baris [20], program membuat nilai seed tetap untuk random generator. Penggunaan seed dilakukan agar hasil pengacakan pada average case tetap

konsisten setiap kali program dijalankan. Dengan begitu, hasil pengujian dapat dibandingkan dengan lebih adil karena target acak yang dipilih tidak berubah-ubah.

Pada baris [22], program membuat alias `SearchFn` menggunakan `std::function`. Tujuan bagian ini adalah agar fungsi pencarian dapat diperlakukan seperti data biasa dan disimpan di dalam vector. Cara ini membuat program lebih fleksibel karena linear search dan binary search dapat diproses menggunakan mekanisme yang sama tanpa perlu menulis kode terpisah untuk masing-masing algoritma.

Pada baris [24]-[27], program membuat kumpulan algoritma yang akan diuji. Setiap pasangan data berisi nama algoritma dan fungsi pencarian yang sesuai. Dengan cara ini, program dapat melakukan pengujian terhadap beberapa algoritma secara otomatis menggunakan loop. Proses berpikir pada bagian ini adalah mengurangi pengulangan kode agar program lebih singkat dan lebih mudah dikembangkan apabila nanti ingin menambahkan algoritma lain.

Fungsi utama program dimulai pada baris [29]. Pada baris [30]-[33], program menampilkan judul program ke layar agar output lebih mudah dibaca dan terlihat rapi.

Pada baris [35], program mulai melakukan loop untuk setiap algoritma yang ada di dalam kumpulan algoritma. Program mengambil nama algoritma dan fungsi pencariannya sekaligus. Cara ini memungkinkan linear search dan binary search diuji menggunakan proses yang sama.

Pada baris [36] dan [37], program menampilkan nama algoritma yang sedang diuji lalu memanggil fungsi untuk menampilkan header tabel hasil pengujian.

Pada baris [39], program memeriksa apakah algoritma yang sedang digunakan adalah binary search. Pemeriksaan ini penting karena best case binary search berbeda dengan linear search. Binary search memiliki best case ketika target berada di tengah data, sedangkan linear search memiliki best case ketika target berada di awal data.

Pada baris [41], program mulai melakukan loop untuk setiap ukuran data yang ada pada `N_VALUES`. Kemudian pada baris [42], program membuat data

terurut sesuai ukuran yang sedang diuji. Data dibuat terurut karena binary search hanya dapat bekerja dengan benar jika data dalam keadaan terurut.

Pada baris [44], program kembali melakukan loop untuk setiap jenis kasus pengujian. Kemudian pada baris [45], program menentukan target pencarian menggunakan fungsi generator target berdasarkan jenis kasus yang dipilih.

Pada baris [47]-[50], program membuat objek Metrics untuk menyimpan hasil pengujian. Setelah itu program mengisi nama algoritma, jenis kasus, dan ukuran data. Langkah ini penting agar setiap hasil pengujian memiliki informasi lengkap ketika ditampilkan ke tabel.

Pada baris [52], program menjalankan algoritma pencarian menggunakan fungsi yang sedang dipilih. Fungsi tersebut menerima data, target, dan objek metrics sehingga hasil pencarian dapat langsung disimpan. Setelah proses pencarian selesai, pada baris [53] program menampilkan hasil pengujian menggunakan fungsi reporter.

Pada baris [56], program mencetak garis pemisah setelah seluruh kasus untuk satu ukuran data selesai diuji. Hal ini dilakukan agar tabel output lebih rapi dan lebih mudah dibaca. Pada baris [60]-[61], program menampilkan tulisan bahwa proses pengujian selesai lalu mengembalikan nilai 0.

Linear Search

Linear search adalah algoritma pencarian yang bekerja dengan cara memeriksa data satu per satu mulai dari elemen pertama hingga data ditemukan atau seluruh data selesai dicek. Dari hasil pengujian yang diperoleh, terlihat bahwa jumlah perbandingan dan waktu eksekusi sangat dipengaruhi oleh posisi data yang dicari. Semakin jauh posisi data dari awal array, maka semakin banyak proses pemeriksaan yang dilakukan. Karena itu, linear search memiliki tiga kondisi kompleksitas waktu yaitu best case, average case, dan worst case.

Pada kondisi **best case**, data yang dicari berada tepat di posisi pertama. Berdasarkan hasil pengujian, untuk semua ukuran data seperti 1000, 10000, 100000, hingga 1000000, jumlah perbandingan selalu bernilai 1. Hal ini terjadi karena algoritma langsung menemukan data pada pemeriksaan pertama sehingga tidak perlu memeriksa elemen lainnya. Waktu eksekusi juga sangat kecil dan hampir tidak berubah walaupun jumlah data semakin besar. Dari kondisi ini dapat dibuktikan bahwa waktu pencarian tidak dipengaruhi oleh ukuran data karena prosesnya selalu hanya satu langkah. Oleh sebab itu kompleksitas waktunya adalah:

$$O(1)$$

Kompleksitas ini konstan karena berapapun jumlah data, proses pencarian tetap hanya membutuhkan satu kali pemeriksaan apabila data berada di posisi pertama.

Pada kondisi **average case**, data yang dicari berada di sekitar tengah kumpulan data. Dari hasil pengujian terlihat bahwa untuk $N = 1000$, jumlah perbandingan sekitar 375, kemudian meningkat menjadi 3746 saat $N = 10000$, lalu 37455 saat $N = 100000$, dan 374541 saat $N = 1000000$. Dari data tersebut terlihat bahwa semakin besar ukuran data, maka jumlah pemeriksaan juga ikut bertambah secara sebanding. Hal ini terjadi karena linear search harus memeriksa elemen satu per satu hingga mencapai posisi target. Walaupun target tidak selalu tepat di tengah, rata-rata jumlah pemeriksaan tetap mendekati sebagian besar data. Karena pertumbuhan jumlah langkah mengikuti pertumbuhan jumlah data, maka kompleksitas waktunya adalah:

$$O(n)$$

Huruf n menunjukkan jumlah data. Jika jumlah data bertambah dua kali lipat, maka jumlah pemeriksaan juga akan bertambah mendekati dua kali lipat. Oleh

karena itu, waktu pencarian pada algoritma linear search akan semakin lama ketika jumlah data semakin banyak.

Pada kondisi **worst case**, data yang dicari tidak ditemukan atau berada di posisi terakhir. Berdasarkan hasil pengujian terlihat bahwa jumlah perbandingan selalu sama dengan jumlah data. Saat $N = 1000$, jumlah perbandingan mencapai 1000. Saat $N = 10000$, jumlah perbandingan menjadi 10000, dan terus meningkat hingga 1000000 perbandingan pada data berukuran 1000000. Hal ini membuktikan bahwa algoritma harus memeriksa seluruh elemen satu per satu sebelum menyimpulkan bahwa data tidak ditemukan. Karena jumlah langkah selalu mengikuti banyaknya data, maka kompleksitas waktunya adalah:

$$O(n)$$

Linear search memiliki karakteristik utama yaitu tidak memerlukan sejumlah data yang sudah terurut. Proses pencarian dilakukan dengan cara memeriksa setiap elemen satu per satu mulai dari indeks pertama hingga data ditemukan. Jika terdapat data 40, 10, 25, 5, 30, algoritma tetap dapat mencari nilai 25 tanpa perlu mengurutkan data terlebih dahulu.

Linear search juga termasuk algoritma yang umumnya diimplementasikan secara iteratif. Implementasi iteratif berarti algoritma menggunakan loop seperti for atau while untuk melakukan pemeriksaan data secara berulang. Pada kode program yang digunakan, linear search memakai loop for untuk memeriksa elemen dari awal hingga akhir. Cara ini lebih sederhana, mudah dipahami, dan lebih hemat memori dibandingkan rekursif. Namun sebenarnya linear search juga dapat dibuat dalam bentuk rekursif (fungsi akan memanggil dirinya sendiri) untuk memeriksa elemen berikutnya. Walaupun hasilnya tetap sama, implementasi rekursif biasanya lebih jarang digunakan karena prosesnya lebih panjang dan menggunakan memori tambahan akibat pemanggilan fungsi berulang.

Mekanisme utama linear search dalam melakukan pencarian adalah membandingkan target dengan setiap elemen data secara berurutan. Algoritma dimulai dari indeks pertama lalu melakukan pemeriksaan satu per satu hingga target ditemukan atau seluruh data selesai diperiksa. Jika elemen yang diperiksa sama dengan target, maka pencarian langsung dihentikan dan indeks data disimpan

sebagai hasil pencarian. Namun, jika elemen tidak sama, algoritma akan melanjutkan pemeriksaan ke elemen berikutnya. Sebagai contoh, jika data berisi [12, 7, 20, 35] dan target yang dicari adalah 20, maka algoritma akan memeriksa 12 terlebih dahulu, kemudian 7, lalu 20. Ketika nilai 20 ditemukan, proses pencarian berhenti. Mekanisme ini menyebabkan jumlah pemeriksaan bisa menjadi sangat banyak ketika ukuran data besar atau data yang dicari berada di bagian akhir.

Berdasarkan hasil pengujian, jumlah comparison yang dilakukan sangat bergantung pada posisi target di dalam data. Pada kondisi best case, jumlah comparison selalu bernilai 1 untuk semua ukuran data, baik 1000, 10000, 100000, maupun 1000000. Hal ini terjadi karena target langsung ditemukan pada indeks pertama sehingga algoritma tidak perlu memeriksa elemen lain. Pada kondisi average case, jumlah comparison meningkat menjadi 375, 3746, 37455, dan 374541. Ini menunjukkan bahwa algoritma linear search harus memeriksa cukup banyak elemen sebelum target ditemukan. Di sisi lain, pada kondisi worst case, jumlah comparison selalu sama dengan jumlah data yaitu 1000, 10000, 100000, dan 1000000. Kondisi ini terjadi karena data tidak ditemukan sehingga seluruh elemen harus diperiksa satu per satu hingga akhir.

Nilai `index_found` menunjukkan posisi data yang berhasil ditemukan di dalam array. Jika data ditemukan, maka program akan menyimpan indeks tempat data tersebut berada. Pada best case, nilai `index_found` selalu 0 karena target berada di posisi pertama. Pada average case, indeks yang ditemukan berada di sekitar tengah data seperti 374, 3745, 37454, dan 374540. Hal ini membuktikan bahwa algoritma memerlukan sejumlah pemeriksaan sebelum mencapai target. Sedangkan pada worst case, nilai yang ditampilkan adalah Not Found, yang berarti `found_index = -1`.

Posisi elemen target sangat mempengaruhi jumlah comparison pada linear search. Semakin dekat posisi target dengan awal array, maka semakin sedikit comparison yang diperlukan. Sebaliknya, semakin jauh posisi target dari awal array, maka semakin banyak comparison yang harus dilakukan. Hal ini terlihat jelas pada hasil pengujian. Saat target berada di indeks pertama, algoritma hanya melakukan satu comparison. Namun, ketika target berada di sekitar tengah data, jumlah comparison meningkat cukup besar. Jika target tidak ada atau berada di

bagian paling akhir, maka algoritma harus memeriksa seluruh data. Karena linear search bekerja secara berurutan dari awal hingga akhir, posisi target menjadi faktor utama yang menentukan cepat atau lambatnya proses pencarian.

Ukuran data (n) juga memberikan pengaruh besar terhadap jumlah operasi pada linear search. Dari hasil pengujian terlihat bahwa ketika ukuran data meningkat dari 1000 menjadi 1000000, jumlah comparison dan waktu eksekusi ikut meningkat secara signifikan. Pada average case misalnya, comparison meningkat dari 375 menjadi 374541, sedangkan waktu eksekusi meningkat dari 2.700 us menjadi 2043.900 us. Pada worst case, peningkatan lebih terlihat lagi karena jumlah comparison selalu sama dengan jumlah data. Hal ini menunjukkan bahwa semakin besar ukuran data, maka semakin banyak operasi yang harus dilakukan oleh linear search. Oleh karena itu, linear search cenderung kurang efisien untuk data berukuran sangat besar karena waktu pencarian akan terus bertambah seiring bertambahnya jumlah data yang diperiksa.

Binary Search

Binary search adalah algoritma pencarian yang bekerja dengan cara membagi data menjadi dua bagian secara terus-menerus hingga data yang dicari ditemukan atau ruang pencarian habis. Berbeda dengan linear search yang memeriksa data satu per satu, binary search langsung memeriksa elemen tengah terlebih dahulu. Jika nilai tengah lebih kecil dari target maka pencarian dilanjutkan ke bagian kanan, sedangkan jika nilai tengah lebih besar maka pencarian dilanjutkan ke bagian kiri. Karena area pencarian selalu dibagi dua pada setiap langkah, binary search jauh lebih cepat dibandingkan linear search, terutama pada data berukuran besar. Namun algoritma ini memiliki syarat bahwa data harus sudah terurut sebelum dilakukan pencarian.

Pada kondisi **best case**, data yang dicari langsung berada pada posisi tengah saat pemeriksaan pertama. Dalam kondisi ideal ini, algoritma hanya membutuhkan satu kali pemeriksaan untuk menemukan target. Secara teori, kompleksitas waktunya adalah:

$$O(1)$$

Kompleksitas ini disebut konstan karena proses pencarian dapat selesai hanya dalam satu langkah tanpa dipengaruhi oleh banyaknya data. Walaupun pada hasil pengujian jumlah comparison tidak selalu bernilai 1, hal tersebut terjadi karena target yang digunakan pada pengujian tidak selalu tepat berada di elemen tengah pertama. Contohnya pada data 1000, comparison mencapai 9, dan pada data 1000000, comparison mencapai 19. Namun secara konsep teori, best case binary search tetap dianggap $O(1)$ karena kondisi terbaik terjadi ketika target langsung ditemukan pada pemeriksaan pertama.

Pada kondisi **average case**, target berada di posisi tertentu sehingga algoritma perlu beberapa kali membagi data sebelum target ditemukan. Dari hasil pengujian terlihat bahwa jumlah comparison tetap kecil walaupun ukuran data meningkat sangat besar. Misalnya pada $N = 1000$, comparison hanya 3, sedangkan pada $N = 1000000$, comparison sekitar 15. Hal ini menunjukkan bahwa penambahan jumlah data tidak membuat jumlah langkah meningkat secara drastis. Penyebabnya adalah karena binary search selalu mengurangi area pencarian menjadi setengah pada setiap langkah. Sebagai contoh sederhana, jika terdapat 16

data maka setelah satu langkah ruang pencarian menjadi 8, lalu 4, kemudian 2, dan akhirnya 1. Karena pengurangan dilakukan dengan pembagian dua secara terus-menerus, maka kompleksitas waktunya adalah:

$$O(\log n)$$

Simbol $\log n$ menunjukkan bahwa jumlah langkah bertambah sangat lambat dibandingkan pertambahan jumlah data. Inilah alasan mengapa binary search sangat efisien untuk data berukuran besar.

Pada kondisi **worst case**, data yang dicari tidak ditemukan atau berada pada posisi yang membutuhkan pemeriksaan paling banyak. Walaupun demikian, binary search tetap jauh lebih cepat dibandingkan linear search karena area pencarian terus diperkecil menjadi setengah. Dari hasil pengujian terlihat bahwa untuk $N = 1000$, comparison maksimum hanya 10, sedangkan untuk $N = 1000000$, comparison maksimum sekitar 20. Jumlah ini sangat kecil dibandingkan linear search yang bisa mencapai jutaan comparison pada ukuran data yang sama. Hal ini membuktikan bahwa walaupun ukuran data meningkat sangat besar, jumlah langkah binary search tetap bertambah perlahan. Oleh karena itu kompleksitas waktunya tetap:

$$O(\log n)$$

Binary search memiliki karakteristik utama yaitu hanya dapat digunakan pada data yang sudah terurut. Hal ini karena mekanisme pencariannya bergantung pada proses pembagian data menjadi dua bagian berdasarkan nilai tengah. Jika data belum terurut, maka algoritma tidak dapat menentukan apakah target berada di sebelah kiri atau kanan dari elemen tengah. Sebagai contoh, jika data terurut seperti [10, 20, 30, 40, 50], lalu target yang dicari adalah 40, algoritma dapat langsung membandingkan target dengan nilai tengah dan menentukan arah pencarian berikutnya. Namun, jika data masih acak seperti [30, 10, 50, 20, 40], proses pembagian tersebut tidak akan bekerja dengan benar karena posisi nilai tidak berurutan. Oleh sebab itu, binary search selalu memerlukan data yang sudah diurutkan terlebih dahulu sebelum pencarian dilakukan.

Binary search umumnya diimplementasikan secara iteratif menggunakan `Looping while`, seperti pada kode program yang digunakan dalam pengujian. Pada implementasi iteratif, algoritma terus memperkecil area pencarian dengan

mengubah batas kiri (left) dan batas kanan (right) hingga data ditemukan atau area pencarian habis. Cara ini lebih sederhana dan lebih hemat memori karena tidak memerlukan pemanggilan fungsi berulang. Namun, binary search juga dapat diimplementasikan secara rekursif. Pada implementasi rekursif, fungsi akan memanggil dirinya sendiri untuk memeriksa bagian kiri atau kanan data berdasarkan hasil comparison sebelumnya. Walaupun hasil pencariannya tetap sama, implementasi rekursif biasanya menggunakan memori tambahan karena setiap pemanggilan fungsi disimpan sementara.

Mekanisme utama binary search dalam melakukan pencarian adalah dengan memeriksa elemen tengah dari data yang sudah terurut. Setelah nilai tengah diperiksa, algoritma akan membandingkan nilai tersebut dengan target yang dicari. Jika nilai tengah sama dengan target, maka pencarian selesai. Jika target lebih kecil dari nilai tengah, maka pencarian dilanjutkan hanya pada bagian kiri data. Sebaliknya, jika target lebih besar dari nilai tengah, maka pencarian dilanjutkan hanya pada bagian kanan data. Proses ini terus dilakukan berulang kali hingga target ditemukan atau area pencarian menjadi kosong. Sebagai contoh, jika data berisi [10, 20, 30, 40, 50, 60, 70] dan target yang dicari adalah 60, maka algoritma pertama kali memeriksa nilai tengah yaitu 40. Karena 60 lebih besar dari 40, pencarian dilanjutkan ke bagian kanan. Kemudian nilai tengah berikutnya diperiksa hingga akhirnya 60 ditemukan. Mekanisme pembagian area pencarian menjadi dua bagian inilah yang membuat binary search jauh lebih cepat dibandingkan linear search, terutama pada data berukuran besar.

Algoritma binary search melakukan pencarian dengan cara membagi area pencarian menjadi dua bagian secara terus-menerus hingga data ditemukan atau area pencarian habis. Karena area pencarian selalu diperkecil, jumlah comparison yang dilakukan jauh lebih sedikit dibandingkan linear search. Pada kondisi best case, jumlah comparison berada pada angka 9, 13, 16, dan 19 untuk ukuran data 1000, 10000, 100000, dan 1000000. Pada average case, jumlah comparison berada pada angka 3, 13, 16, dan 15. Sedangkan pada worst case, jumlah comparison hanya mencapai 10, 14, 17, dan 20. Dari hasil tersebut terlihat bahwa walaupun ukuran data meningkat sangat besar, jumlah comparison tidak bertambah secara drastis. Hal ini terjadi karena setiap langkah binary search langsung mengurangi area

pencarian menjadi setengah, sehingga jumlah pemeriksaan menjadi jauh lebih sedikit dibandingkan algoritma yang memeriksa data satu per satu.

Nilai `index_found` menunjukkan posisi data yang berhasil ditemukan. Pada kondisi best case, nilai `index_found` berada pada indeks tengah data seperti 500, 5000, 50000, dan 500000. Hal ini menunjukkan bahwa target ditemukan setelah beberapa proses pembagian data dilakukan. Pada average case, nilai indeks yang ditemukan berada di sekitar 374, 3745, 37454, dan 374540. Nilai tersebut menunjukkan posisi target di dalam array setelah algoritma melakukan beberapa comparison sebelum data ditemukan. Sedangkan pada worst case, hasil yang ditampilkan adalah Not Found, yang berarti nilai `found_index` = -1. Nilai -1 digunakan sebagai tanda bahwa target tidak ditemukan setelah seluruh proses pembagian area pencarian selesai dilakukan.

Posisi elemen target juga mempengaruhi jumlah comparison pada binary search, walaupun pengaruhnya tidak sebesar pada linear search. Jika target berada dekat dengan nilai tengah yang pertama diperiksa, maka jumlah comparison akan lebih sedikit karena data dapat ditemukan lebih cepat. Namun, jika target berada lebih jauh dari posisi tengah atau bahkan tidak ada di dalam data, maka algoritma perlu melakukan lebih banyak pembagian area pencarian. Walaupun demikian, jumlah langkah yang diperlukan tetap relatif kecil karena setiap comparison langsung mengurangi setengah area pencarian.

Ukuran data (n) memberikan pengaruh terhadap jumlah operasi pada binary search, tetapi peningkatannya terjadi secara perlahan. Dari hasil pengujian terlihat bahwa ketika ukuran data meningkat dari 1000 menjadi 1000000, jumlah comparison hanya berubah dari sekitar 10 menjadi sekitar 20. Perubahan ini sangat kecil dibandingkan linear search yang jumlah comparison-nya bisa meningkat hingga jutaan pemeriksaan. Hal ini terjadi karena binary search tidak memeriksa seluruh data satu per satu, melainkan terus membagi area pencarian menjadi dua bagian. Sebagai contoh sederhana, data berukuran 1000000 tidak perlu diperiksa satu juta kali, tetapi cukup sekitar 20 langkah untuk mempersempit area pencarian hingga target ditemukan atau dipastikan tidak ada.

Berdasarkan hasil pengujian dari Linear Search dan Binary Search yang diperoleh, binary search memiliki waktu eksekusi yang jauh lebih kecil dibandingkan linear search pada kondisi data yang sama. Pada linear search, waktu eksekusi meningkat cukup besar ketika ukuran data bertambah. Sebagai contoh, pada ukuran data 1000, waktu eksekusi worst case masih berada pada kisaran mikrodetik kecil, tetapi ketika ukuran data meningkat menjadi 1000000, waktu eksekusi meningkat hingga ribuan microseconds. Hal ini terjadi karena linear search memeriksa data satu per satu dari awal hingga akhir. Sebaliknya, binary search menunjukkan peningkatan waktu yang jauh lebih kecil walaupun ukuran data meningkat sangat besar. Pada data berukuran 1000000, binary search tetap hanya membutuhkan puluhan comparison dan waktu eksekusi yang relatif rendah. Dari hasil tersebut dapat dilihat bahwa binary search mampu melakukan pencarian lebih cepat karena area pencarian langsung diperkecil menjadi setengah pada setiap langkah.

Untuk jumlah data yang kecil, perbedaan performa antara linear search dan binary search tidak terlalu terlihat jauh. Hal ini karena jumlah data yang diperiksa masih sedikit sehingga linear search masih dapat bekerja cukup cepat. Bahkan dalam beberapa kondisi tertentu, linear search dapat memiliki waktu yang hampir sama ketika target berada di posisi awal data. Namun, ketika jumlah data semakin besar, binary search menjadi jauh lebih efisien dibandingkan linear search. Hal ini terlihat dari jumlah comparison linear search yang dapat mencapai jutaan pemeriksaan, sedangkan binary search tetap hanya membutuhkan belasan hingga puluhan comparison saja. Oleh karena itu, linear search masih cukup layak digunakan pada jumlah data yang kecil atau sederhana, sedangkan binary search lebih cocok digunakan pada jumlah data yang besar karena performanya jauh lebih stabil dan cepat.

Kondisi data juga sangat mempengaruhi performa kedua algoritma. Linear search tidak memerlukan data yang terurut karena algoritma ini hanya memeriksa data satu per satu secara berurutan. Oleh sebab itu, performa linear search pada data acak maupun data terurut tidak memiliki perbedaan yang terlalu besar. Sebaliknya, binary search sangat bergantung pada data yang terurut. Jika data tidak terurut, maka mekanisme pembagian area pencarian tidak dapat bekerja dengan benar

karena algoritma tidak bisa menentukan apakah target berada di bagian kiri atau kanan data tengah. Inilah alasan mengapa sebelum menggunakan binary search biasanya diperlukan proses sorting terlebih dahulu. Walaupun sorting membutuhkan waktu tambahan, binary search tetap lebih menguntungkan untuk jumlah data yang besar karena proses pencariannya jauh lebih cepat setelah data terurut.

Hasil eksperimen juga menunjukkan hubungan yang sesuai dengan teori kompleksitas waktu kedua algoritma. Linear search memiliki kompleksitas:

$$O(n)$$

Hal ini berarti jumlah operasi bertambah sebanding dengan jumlah data. Dari hasil pengujian terlihat bahwa ketika ukuran data meningkat berkali-kali lipat, jumlah comparison dan waktu eksekusi linear search juga meningkat secara besar. Kondisi tersebut membuktikan bahwa linear search memang harus memeriksa banyak elemen ketika ukuran data bertambah. Di sisi lain, binary search memiliki kompleksitas:

$$O(\log n)$$

Kompleksitas ini menunjukkan bahwa jumlah langkah bertambah jauh lebih lambat dibandingkan pertambahan jumlah data. Hasil eksperimen membuktikan hal tersebut karena walaupun ukuran data meningkat dari 1000 menjadi 1000000, selisih dari jumlah comparison binary search sedikit.